Appendix

The following sections contain reference material you may find useful in your Rust journey.

A - Keywords

Appendix A: Keywords

The following lists contain keywords that are reserved for current or future use by the Rust language. As such, they cannot be used as identifiers (except as raw identifiers, as we discuss in the "Raw Identifiers" section). *Identifiers* are names of functions, variables, parameters, struct fields, modules, crates, constants, macros, static values, attributes, types, traits, or lifetimes.

Keywords Currently in Use

The following is a list of keywords currently in use, with their functionality described.

- **`as`** : Perform primitive casting, disambiguate the specific trait containing an item, or rename items in `use` statements.
- **`async`** : Return a `Future` instead of blocking the current thread.
- **`await`** : Suspend execution until the result of a `Future` is ready.
- **`break`** : Exit a loop immediately.
- **`const`** : Define constant items or constant raw pointers.
- **`continue`** : Continue to the next loop iteration.
- **`crate`** : In a module path, refers to the crate root.
- **`dyn`** : Dynamic dispatch to a trait object.
- **`else`** : Fallback for `if` and `if let` control flow constructs.
- **`enum`** : Define an enumeration.
- **`extern`** : Link an external function or variable.
- **`false`** : Boolean false literal.
- **`fn`** : Define a function or the function pointer type.
- **`for`** : Loop over items from an iterator, implement a trait, or specify a higher ranked lifetime.
- **`if`** : Branch based on the result of a conditional expression.
- **`impl`** : Implement inherent or trait functionality.
- **`in`** : Part of `for` loop syntax.
- **`let`** : Bind a variable.
- **`loop`** : Loop unconditionally.
- **`match`** : Match a value to patterns.
- **`mod`** : Define a module.
- **`move`** : Make a closure take ownership of all its captures.
- **`mut`** : Denote mutability in references, raw pointers, or pattern bindings.

- `pub` : Denote public visibility in struct fields, `impl` blocks, or modules.
- `ref` : Bind by reference.
- `return` : Return from function.
- `Self` : A type alias for the type we are defining or implementing.
- `self` : Method subject or current module.
- `static` : Global variable or lifetime lasting the entire program execution.
- `struct` : Define a structure.
- `super` : Parent module of the current module.
- `trait` : Define a trait.
- `true` : Boolean true literal.
- `type` : Define a type alias or associated type.
- `union` : Define a union; is a keyword only when used in a union declaration.
- `unsafe` : Denote unsafe code, functions, traits, or implementations.
- `use` : Bring symbols into scope.
- `where` : Denote clauses that constrain a type.
- `while` : Loop conditionally based on the result of an expression.

Keywords Reserved for Future Use

The following keywords do not yet have any functionality but are reserved by Rust for potential future use:

- `abstract`
- `become`
- `box`
- `do`
- `final`
- `gen`
- `macro`
- `override`
- `priv`
- `try`
- `typeof`
- `unsized`
- `virtual`
- `yield`

Raw Identifiers

Raw identifiers are the syntax that lets you use keywords where they wouldn't normally be allowed. You use a raw identifier by prefixing a keyword with `r#` .

For example, `match` is a keyword. If you try to compile the following function that uses `match` as its name:

Filename: src/main.rs

```
fn match(needle: &str, haystack: &str) -> bool {
    haystack.contains(needle)
}
```

you'll get this error:

```
error: expected identifier, found keyword `match`
 --> src/main.rs:4:4
  |
4 | fn match(needle: &str, haystack: &str) -> bool {
  |    ^^^^^ expected identifier, found keyword
```

The error shows that you can't use the keyword `match` as the function identifier. To use `match` as a function name, you need to use the raw identifier syntax, like this:

Filename: src/main.rs

```
fn r#match(needle: &str, haystack: &str) -> bool {
    haystack.contains(needle)
}

fn main() {
    assert!(r#match("foo", "foobar"));
}
```

This code will compile without any errors. Note the `r#` prefix on the function name in its definition as well as where the function is called in `main`.

Raw identifiers allow you to use any word you choose as an identifier, even if that word happens to be a reserved keyword. This gives us more freedom to choose identifier names, as well as lets us integrate with programs written in a language where these words aren't keywords. In addition, raw identifiers allow you to use libraries written in a different Rust edition than your crate uses. For example, `try` isn't a keyword in the 2015 edition but is in the 2018, 2021, and 2024 editions. If you depend on a library that is written using the 2015 edition and has a `try` function, you'll need to use the raw identifier syntax, `r#try` in this case, to call that function from your code on later editions. See Appendix E for more information on editions.

B - Operators and Symbols

Appendix B: Operators and Symbols

This appendix contains a glossary of Rust's syntax, including operators and other symbols that appear by themselves or in the context of paths, generics, trait bounds, macros, attributes, comments, tuples, and brackets.

Operators

Table B-1 contains the operators in Rust, an example of how the operator would appear in context, a short explanation, and whether that operator is overloadable. If an operator is overloadable, the relevant trait to use to overload that operator is listed.

Table B-1: Operators

Operator	Example	Explanation	Overloadable?
`!`	`ident!(...)`, `ident!{...}`, `ident![...]`	Macro expansion	
`!`	`!expr`	Bitwise or logical complement	`Not`
`!=`	`expr != expr`	Nonequality comparison	`PartialEq`
`%`	`expr % expr`	Arithmetic remainder	`Rem`
`%=`	`var %= expr`	Arithmetic remainder and assignment	`RemAssign`
`&`	`&expr`, `&mut expr`	Borrow	
`&`	`&type`, `&mut type`, `&'a type`, `&'a mut type`	Borrowed pointer type	
`&`	`expr & expr`	Bitwise AND	`BitAnd`
`&=`	`var &= expr`	Bitwise AND and assignment	`BitAndAssign`
`&&`	`expr && expr`	Short-circuiting logical AND	
`*`	`expr * expr`	Arithmetic multiplication	`Mul`
`*=`	`var *= expr`	Arithmetic multiplication and assignment	`MulAssign`

Operator	Example	Explanation	Overloadable?
`*`	`*expr`	Dereference	`Deref`
`*`	`*const type`, `*mut type`	Raw pointer	
`+`	`trait + trait`, `'a + trait`	Compound type constraint	
`+`	`expr + expr`	Arithmetic addition	`Add`
`+=`	`var += expr`	Arithmetic addition and assignment	`AddAssign`
`,`	`expr, expr`	Argument and element separator	
`-`	`- expr`	Arithmetic negation	`Neg`
`-`	`expr - expr`	Arithmetic subtraction	`Sub`
`-=`	`var -= expr`	Arithmetic subtraction and assignment	`SubAssign`
`->`	`fn(...) -> type`, `|…| -> type`	Function and closure return type	
`.`	`expr.ident`	Field access	
`.`	`expr.ident(expr, ...)`	Method call	
`.`	`expr.0`, `expr.1`, and so on	Tuple indexing	
`..`	`..`, `expr..`, `..expr`, `expr..expr`	Right-exclusive range literal	`PartialOrd`
`..=`	`..=expr`, `expr..=expr`	Right-inclusive range literal	`PartialOrd`
`..`	`..expr`	Struct literal update syntax	
`..`	`variant(x, ..)`, `struct_type { x, .. }`	“And the rest” pattern binding	
`...`	`expr...expr`	(Deprecated, use `..=` instead) In a pattern: inclusive range pattern	
`/`	`expr / expr`	Arithmetic division	`Div`
`/=`	`var /= expr`	Arithmetic division and assignment	`DivAssign`
`:`	`pat: type`, `ident: type`	Constraints	

Operator	Example	Explanation	Overloadable?
`:`	`ident: expr`	Struct field initializer	
`:`	`'a: loop {...}`	Loop label	
`;`	`expr;`	Statement and item terminator	
`;`	`[...; len]`	Part of fixed-size array syntax	
`<<`	`expr << expr`	Left-shift	`Shl`
`<<=`	`var <<= expr`	Left-shift and assignment	`ShlAssign`
`<`	`expr < expr`	Less than comparison	`PartialOrd`
`<=`	`expr <= expr`	Less than or equal to comparison	`PartialOrd`
`=`	`var = expr, ident = type`	Assignment/equivalence	
`==`	`expr == expr`	Equality comparison	`PartialEq`
`=>`	`pat => expr`	Part of match arm syntax	
`>`	`expr > expr`	Greater than comparison	`PartialOrd`
`>=`	`expr >= expr`	Greater than or equal to comparison	`PartialOrd`
`>>`	`expr >> expr`	Right-shift	`Shr`
`>>=`	`var >>= expr`	Right-shift and assignment	`ShrAssign`
`@`	`ident @ pat`	Pattern binding	
`^`	`expr ^ expr`	Bitwise exclusive OR	`BitXor`
`^=`	`var ^= expr`	Bitwise exclusive OR and assignment	`BitXorAssign`
`|`	`pat | pat`	Pattern alternatives	
`|`	`expr | expr`	Bitwise OR	`BitOr`
`|=`	`var |= expr`	Bitwise OR and assignment	`BitOrAssign`
`||`	`expr || expr`	Short-circuiting logical OR	
`?`	`expr?`	Error propagation	

Non-operator Symbols

The following tables contain all symbols that don't function as operators; that is, they don't behave like a function or method call.

Table B-2 shows symbols that appear on their own and are valid in a variety of locations.

Table B-2: Stand-alone Syntax

Symbol	Explanation
`'ident`	Named lifetime or loop label
Digits immediately followed by `u8`, `i32`, `f64`, `usize`, and so on	Numeric literal of specific type
`"..."`	String literal
`r"..."`, `r#"..."#`, `r##"..."##`, and so on	Raw string literal; escape characters not processed
`b"..."`	Byte string literal; constructs an array of bytes instead of a string
`br"..."`, `br#"..."#`, `br##"..."##`, and so on	Raw byte string literal; combination of raw and byte string literal
`'...'`	Character literal
`b'...'`	ASCII byte literal
`|…| expr`	Closure
`!`	Always-empty bottom type for diverging functions
`_`	“Ignored” pattern binding; also used to make integer literals readable

Table B-3 shows symbols that appear in the context of a path through the module hierarchy to an item.

Table B-3: Path-Related Syntax

Symbol	Explanation
`ident::ident`	Namespace path
`::path`	Path relative to the crate root (that is, an explicitly absolute path)
`self::path`	Path relative to the current module (that is, an explicitly relative path)
`super::path`	Path relative to the parent of the current module
`type::ident`, `<type as trait>::ident`	Associated constants, functions, and types
`<type>::...`	Associated item for a type that cannot be directly named (for example, `<&T>::...`, `<[T]>::...`, and so on)
`trait::method(...)`	Disambiguating a method call by naming the trait that defines it

Symbol	Explanation
`type::method(...)`	Disambiguating a method call by naming the type for which it's defined
`<type as trait>::method(...)`	Disambiguating a method call by naming the trait and type

Table B-4 shows symbols that appear in the context of using generic type parameters.

Table B-4: Generics

Symbol	Explanation
`path<...>`	Specifies parameters to a generic type in a type (for example, `Vec<u8>`)
`path::<...>`, `method::<...>`	Specifies parameters to a generic type, function, or method in an expression; often referred to as *turbofish* (for example, `"42".parse::<i32>()`)
`fn ident<...> ...`	Define generic function
`struct ident<...> ...`	Define generic structure
`enum ident<...> ...`	Define generic enumeration
`impl<...> ...`	Define generic implementation
`for<...> type`	Higher ranked lifetime bounds
`type<ident=type>`	A generic type where one or more associated types have specific assignments (for example, `Iterator<Item=T>`)

Table B-5 shows symbols that appear in the context of constraining generic type parameters with trait bounds.

Table B-5: Trait Bound Constraints

Symbol	Explanation
`T: U`	Generic parameter `T` constrained to types that implement `U`
`T: 'a`	Generic type `T` must outlive lifetime `'a` (meaning the type cannot transitively contain any references with lifetimes shorter than `'a`)
`T: 'static`	Generic type `T` contains no borrowed references other than `'static` ones
`'b: 'a`	Generic lifetime `'b` must outlive lifetime `'a`
`T: ?Sized`	Allow generic type parameter to be a dynamically sized type
`'a + trait`, `trait + trait`	Compound type constraint

Table B-6 shows symbols that appear in the context of calling or defining macros and specifying attributes on an item.

Table B-6: Macros and Attributes

Symbol	Explanation
`#[meta]`	Outer attribute
`#![meta]`	Inner attribute
`$ident`	Macro substitution
`$ident:kind`	Macro metavariable
`$(...)...`	Macro repetition
`ident!(...)`, `ident!{...}`, `ident![...]`	Macro invocation

Table B-7 shows symbols that create comments.

Table B-7: Comments

Symbol	Explanation
`//`	Line comment
`//!`	Inner line doc comment
`///`	Outer line doc comment
`/*...*/`	Block comment
`/*!...*/`	Inner block doc comment
`/**...*/`	Outer block doc comment

Table B-8 shows the contexts in which parentheses are used.

Table B-8: Parentheses

Symbol	Explanation
`()`	Empty tuple (aka unit), both literal and type
`(expr)`	Parenthesized expression
`(expr,)`	Single-element tuple expression
`(type,)`	Single-element tuple type
`(expr, ...)`	Tuple expression
`(type, ...)`	Tuple type
`expr(expr, ...)`	Function call expression; also used to initialize tuple `struct`s and tuple `enum` variants

Table B-9 shows the contexts in which curly brackets are used.

Table B-9: Curly Brackets

Context	Explanation
`{...}`	Block expression
`Type {...}`	Struct literal

Table B-10 shows the contexts in which square brackets are used.

Table B-10: Square Brackets

Context	Explanation
`[...]`	Array literal
`[expr; len]`	Array literal containing `len` copies of `expr`
`[type; len]`	Array type containing `len` instances of `type`
`expr[expr]`	Collection indexing; overloadable (`Index` , `IndexMut`)
`expr[..]` , `expr[a..]` , `expr[..b]` , `expr[a..b]`	Collection indexing pretending to be collection slicing, using `Range` , `RangeFrom` , `RangeTo` , or `RangeFull` as the “index”

C - Derivable Traits

Appendix C: Derivable Traits

In various places in the book, we've discussed the `derive` attribute, which you can apply to a struct or enum definition. The `derive` attribute generates code that will implement a trait with its own default implementation on the type you've annotated with the `derive` syntax.

In this appendix, we provide a reference of all the traits in the standard library that you can use with `derive`. Each section covers:

- What operators and methods deriving this trait will enable
- What the implementation of the trait provided by `derive` does
- What implementing the trait signifies about the type
- The conditions in which you're allowed or not allowed to implement the trait
- Examples of operations that require the trait

If you want different behavior from that provided by the `derive` attribute, consult the standard library documentation for each trait for details on how to manually implement them.

The traits listed here are the only ones defined by the standard library that can be implemented on your types using `derive`. Other traits defined in the standard library don't have sensible default behavior, so it's up to you to implement them in the way that makes sense for what you're trying to accomplish.

An example of a trait that can't be derived is `Display`, which handles formatting for end users. You should always consider the appropriate way to display a type to an end user. What parts of the type should an end user be allowed to see? What parts would they find relevant? What format of the data would be most relevant to them? The Rust compiler doesn't have this insight, so it can't provide appropriate default behavior for you.

The list of derivable traits provided in this appendix is not comprehensive: Libraries can implement `derive` for their own traits, making the list of traits you can use `derive` with truly open ended. Implementing `derive` involves using a procedural macro, which is covered in the "Custom `derive` Macros" section in Chapter 20.

Debug for Programmer Output

The `Debug` trait enables debug formatting in format strings, which you indicate by adding `:?` within `{}` placeholders.

The `Debug` trait allows you to print instances of a type for debugging purposes, so you and other programmers using your type can inspect an instance at a particular point in a program's execution.

The `Debug` trait is required, for example, in the use of the `assert_eq!` macro. This macro prints the values of instances given as arguments if the equality assertion fails so that programmers can see why the two instances weren't equal.

`PartialEq` and `Eq` for Equality Comparisons

The `PartialEq` trait allows you to compare instances of a type to check for equality and enables use of the `==` and `!=` operators.

Deriving `PartialEq` implements the `eq` method. When `PartialEq` is derived on structs, two instances are equal only if *all* fields are equal, and the instances are not equal if *any* fields are not equal. When derived on enums, each variant is equal to itself and not equal to the other variants.

The `PartialEq` trait is required, for example, with the use of the `assert_eq!` macro, which needs to be able to compare two instances of a type for equality.

The `Eq` trait has no methods. Its purpose is to signal that for every value of the annotated type, the value is equal to itself. The `Eq` trait can only be applied to types that also implement `PartialEq`, although not all types that implement `PartialEq` can implement `Eq`. One example of this is floating-point number types: The implementation of floating-point numbers states that two instances of the not-a-number (`NaN`) value are not equal to each other.

An example of when `Eq` is required is for keys in a `HashMap<K, V>` so that the `HashMap<K, V>` can tell whether two keys are the same.

`PartialOrd` and `Ord` for Ordering Comparisons

The `PartialOrd` trait allows you to compare instances of a type for sorting purposes. A type that implements `PartialOrd` can be used with the `<`, `>`, `<=`, and `>=` operators. You can only apply the `PartialOrd` trait to types that also implement `PartialEq`.

Deriving `PartialOrd` implements the `partial_cmp` method, which returns an `Option<Ordering>` that will be `None` when the values given don't produce an ordering. An example of a value that doesn't produce an ordering, even though most values of that type can be compared, is the `NaN` floating point value. Calling `partial_cmp` with any floating-point number and the `NaN` floating-point value will return `None`.

When derived on structs, `PartialOrd` compares two instances by comparing the value in each field in the order in which the fields appear in the struct definition. When derived on enums, variants of the enum declared earlier in the enum definition are considered less than the variants listed later.

The `PartialOrd` trait is required, for example, for the `gen_range` method from the `rand` crate that generates a random value in the range specified by a range expression.

The `Ord` trait allows you to know that for any two values of the annotated type, a valid ordering will exist. The `Ord` trait implements the `cmp` method, which returns an `Ordering` rather than an `Option<Ordering>` because a valid ordering will always be possible. You can only apply the `Ord` trait to types that also implement `PartialOrd` and `Eq` (and `Eq` requires `PartialEq`). When derived on structs and enums, `cmp` behaves the same way as the derived implementation for `partial_cmp` does with `PartialOrd`.

An example of when `Ord` is required is when storing values in a `BTreeSet<T>`, a data structure that stores data based on the sort order of the values.

`Clone` and `Copy` for Duplicating Values

The `Clone` trait allows you to explicitly create a deep copy of a value, and the duplication process might involve running arbitrary code and copying heap data. See the "Variables and Data Interacting with Clone" section in Chapter 4 for more information on `Clone`.

Deriving `Clone` implements the `clone` method, which when implemented for the whole type, calls `clone` on each of the parts of the type. This means all the fields or values in the type must also implement `Clone` to derive `Clone`.

An example of when `Clone` is required is when calling the `to_vec` method on a slice. The slice doesn't own the type instances it contains, but the vector returned from `to_vec` will need to own its instances, so `to_vec` calls `clone` on each item. Thus, the type stored in the slice must implement `Clone`.

The `Copy` trait allows you to duplicate a value by only copying bits stored on the stack; no arbitrary code is necessary. See the "Stack-Only Data: Copy" section in Chapter 4 for more information on `Copy`.

The `Copy` trait doesn't define any methods to prevent programmers from overloading those methods and violating the assumption that no arbitrary code is being run. That way, all programmers can assume that copying a value will be very fast.

You can derive `Copy` on any type whose parts all implement `Copy`. A type that implements `Copy` must also implement `Clone` because a type that implements `Copy` has a trivial implementation of `Clone` that performs the same task as `Copy`.

The `Copy` trait is rarely required; types that implement `Copy` have optimizations available, meaning you don't have to call `clone`, which makes the code more concise.

Everything possible with `Copy` you can also accomplish with `Clone`, but the code might be slower or have to use `clone` in places.

`Hash` for Mapping a Value to a Value of Fixed Size

The `Hash` trait allows you to take an instance of a type of arbitrary size and map that instance to a value of fixed size using a hash function. Deriving `Hash` implements the `hash` method. The derived implementation of the `hash` method combines the result of calling `hash` on each of the parts of the type, meaning all fields or values must also implement `Hash` to derive `Hash`.

An example of when `Hash` is required is in storing keys in a `HashMap<K, V>` to store data efficiently.

`Default` for Default Values

The `Default` trait allows you to create a default value for a type. Deriving `Default` implements the `default` function. The derived implementation of the `default` function calls the `default` function on each part of the type, meaning all fields or values in the type must also implement `Default` to derive `Default`.

The `Default::default` function is commonly used in combination with the struct update syntax discussed in the "Creating Instances from Other Instances with Struct Update Syntax" section in Chapter 5. You can customize a few fields of a struct and then set and use a default value for the rest of the fields by using `..Default::default()`.

The `Default` trait is required when you use the method `unwrap_or_default` on `Option<T>` instances, for example. If the `Option<T>` is `None`, the method `unwrap_or_default` will return the result of `Default::default` for the type `T` stored in the `Option<T>`.

D - Useful Development Tools

Appendix D: Useful Development Tools

In this appendix, we talk about some useful development tools that the Rust project provides. We'll look at automatic formatting, quick ways to apply warning fixes, a linter, and integrating with IDEs.

Automatic Formatting with `rustfmt`

The `rustfmt` tool reformats your code according to the community code style. Many collaborative projects use `rustfmt` to prevent arguments about which style to use when writing Rust: Everyone formats their code using the tool.

Rust installations include `rustfmt` by default, so you should already have the programs `rustfmt` and `cargo-fmt` on your system. These two commands are analogous to `rustc` and `cargo` in that `rustfmt` allows finer grained control and `cargo-fmt` understands conventions of a project that uses Cargo. To format any Cargo project, enter the following:

```
$ cargo fmt
```

Running this command reformats all the Rust code in the current crate. This should only change the code style, not the code semantics. For more information on `rustfmt`, see its documentation.

Fix Your Code with `rustfix`

The `rustfix` tool is included with Rust installations and can automatically fix compiler warnings that have a clear way to correct the problem that's likely what you want. You've probably seen compiler warnings before. For example, consider this code:

Filename: src/main.rs

```
fn main() {
    let mut x = 42;
    println!("{x}");
}
```

Here, we're defining the variable `x` as mutable, but we never actually mutate it. Rust warns us about that:

```
$ cargo build
   Compiling myprogram v0.1.0 (file:///projects/myprogram)
warning: variable does not need to be mutable
 --> src/main.rs:2:9
  |
2 |     let mut x = 0;
  |         ----^
  |         |
  |         help: remove this `mut`
  |
  = note: `#[warn(unused_mut)]` on by default
```

The warning suggests that we remove the `mut` keyword. We can automatically apply that suggestion using the `rustfix` tool by running the command `cargo fix`:

```
$ cargo fix
    Checking myprogram v0.1.0 (file:///projects/myprogram)
      Fixing src/main.rs (1 fix)
    Finished dev [unoptimized + debuginfo] target(s) in 0.59s
```

When we look at *src/main.rs* again, we'll see that `cargo fix` has changed the code:

Filename: src/main.rs

```
fn main() {
    let x = 42;
    println!("{x}");
}
```

The variable `x` is now immutable, and the warning no longer appears.

You can also use the `cargo fix` command to transition your code between different Rust editions. Editions are covered in Appendix E.

More Lints with Clippy

The Clippy tool is a collection of lints to analyze your code so that you can catch common mistakes and improve your Rust code. Clippy is included with standard Rust installations.

To run Clippy's lints on any Cargo project, enter the following:

```
$ cargo clippy
```

For example, say you write a program that uses an approximation of a mathematical constant, such as pi, as this program does:

Filename: src/main.rs

```
fn main() {
    let x = 3.1415;
    let r = 8.0;
    println!("the area of the circle is {}", x * r * r);
}
```

Running `cargo clippy` on this project results in this error:

```
error: approximate value of `f{32, 64}::consts::PI` found
 --> src/main.rs:2:13
  |
2 |     let x = 3.1415;
  |             ^^^^^^
  |
  = note: `#[deny(clippy::approx_constant)]` on by default
  = help: consider using the constant directly
  = help: for further information visit https://rust-lang.github.io/rust-
clippy/master/index.html#approx_constant
```

This error lets you know that Rust already has a more precise `PI` constant defined, and that your program would be more correct if you used the constant instead. You would then change your code to use the `PI` constant.

The following code doesn't result in any errors or warnings from Clippy:

Filename: src/main.rs

```
fn main() {
    let x = std::f64::consts::PI;
    let r = 8.0;
    println!("the area of the circle is {}", x * r * r);
}
```

For more information on Clippy, see its documentation.

IDE Integration Using `rust-analyzer`

To help with IDE integration, the Rust community recommends using `rust-analyzer`. This tool is a set of compiler-centric utilities that speak Language Server Protocol, which is a specification for IDEs and programming languages to communicate with each other. Different clients can use `rust-analyzer`, such as the Rust analyzer plug-in for Visual Studio Code.

Visit the `rust-analyzer` project's home page for installation instructions, then install the language server support in your particular IDE. Your IDE will gain capabilities such as autocompletion, jump to definition, and inline errors.

E - Editions

Appendix E: Editions

In Chapter 1, you saw that `cargo new` adds a bit of metadata to your *Cargo.toml* file about an edition. This appendix talks about what that means!

The Rust language and compiler have a six-week release cycle, meaning users get a constant stream of new features. Other programming languages release larger changes less often; Rust releases smaller updates more frequently. After a while, all of these tiny changes add up. But from release to release, it can be difficult to look back and say, “Wow, between Rust 1.10 and Rust 1.31, Rust has changed a lot!”

Every three years or so, the Rust team produces a new Rust *edition*. Each edition brings together the features that have landed into a clear package with fully updated documentation and tooling. New editions ship as part of the usual six-week release process.

Editions serve different purposes for different people:

- For active Rust users, a new edition brings together incremental changes into an easy-to-understand package.
- For non-users, a new edition signals that some major advancements have landed, which might make Rust worth another look.
- For those developing Rust, a new edition provides a rallying point for the project as a whole.

At the time of this writing, four Rust editions are available: Rust 2015, Rust 2018, Rust 2021, and Rust 2024. This book is written using Rust 2024 edition idioms.

The `edition` key in *Cargo.toml* indicates which edition the compiler should use for your code. If the key doesn’t exist, Rust uses `2015` as the edition value for backward compatibility reasons.

Each project can opt in to an edition other than the default 2015 edition. Editions can contain incompatible changes, such as including a new keyword that conflicts with identifiers in code. However, unless you opt in to those changes, your code will continue to compile even as you upgrade the Rust compiler version you use.

All Rust compiler versions support any edition that existed prior to that compiler’s release, and they can link crates of any supported editions together. Edition changes only affect the way the compiler initially parses code. Therefore, if you’re using Rust 2015 and one of your dependencies uses Rust 2018, your project will compile and be able to use that dependency.

The opposite situation, where your project uses Rust 2018 and a dependency uses Rust 2015, works as well.

To be clear: Most features will be available on all editions. Developers using any Rust edition will continue to see improvements as new stable releases are made. However, in some cases, mainly when new keywords are added, some new features might only be available in later editions. You will need to switch editions if you want to take advantage of such features.

For more details, see *The Rust Edition Guide*. This is a complete book that enumerates the differences between editions and explains how to automatically upgrade your code to a new edition via `cargo fix`.

F - Translations of the Book

Appendix F: Translations of the Book

For resources in languages other than English. Most are still in progress; see the Translations label to help or let us know about a new translation!

- Português (BR)
- Português (PT)
- 简体中文: KaiserY/trpl-zh-cn, gnu4cn/rust-lang-Zh_CN
- 正體中文
- Українська
- Español, alternate, Español por RustLangES
- Русский
- 한국어
- 日本語
- Français
- Polski
- Cebuano
- Tagalog
- Esperanto
- ελληνική
- Svenska
- Farsi, Persian (FA)
- Deutsch
- हिंदी
- ไทย
- Danske
- O'zbek
- Tiếng Việt
- Italiano
- বাংলা

G - How Rust is Made and “Nightly Rust”

Appendix G - How Rust is Made and “Nightly Rust”

This appendix is about how Rust is made and how that affects you as a Rust developer.

Stability Without Stagnation

As a language, Rust cares a *lot* about the stability of your code. We want Rust to be a rock-solid foundation you can build on, and if things were constantly changing, that would be impossible. At the same time, if we can’t experiment with new features, we may not find out important flaws until after their release, when we can no longer change things.

Our solution to this problem is what we call “stability without stagnation”, and our guiding principle is this: you should never have to fear upgrading to a new version of stable Rust. Each upgrade should be painless, but should also bring you new features, fewer bugs, and faster compile times.

Choo, Choo! Release Channels and Riding the Trains

Rust development operates on a *train schedule*. That is, all development is done in the main branch of the Rust repository. Releases follow a software release train model, which has been used by Cisco IOS and other software projects. There are three *release channels* for Rust:

- Nightly
- Beta
- Stable

Most Rust developers primarily use the stable channel, but those who want to try out experimental new features may use nightly or beta.

Here’s an example of how the development and release process works: let’s assume that the Rust team is working on the release of Rust 1.5. That release happened in December of 2015, but it will provide us with realistic version numbers. A new feature is added to Rust: a new commit lands on the main branch. Each night, a new nightly version of Rust is produced. Every day is a release day, and these releases are created by our release infrastructure automatically. So as time passes, our releases look like this, once a night:

```
nightly: * - - * - - *
```

Every six weeks, it's time to prepare a new release! The `beta` branch of the Rust repository branches off from the main branch used by nightly. Now, there are two releases:

```
nightly: * - - * - - *
                     |
beta:                *
```

Most Rust users do not use beta releases actively, but test against beta in their CI system to help Rust discover possible regressions. In the meantime, there's still a nightly release every night:

```
nightly: * - - * - - * - - * - - *
                     |
beta:                *
```

Let's say a regression is found. Good thing we had some time to test the beta release before the regression snuck into a stable release! The fix is applied to the main branch, so that nightly is fixed, and then the fix is backported to the `beta` branch, and a new release of beta is produced:

```
nightly: * - - * - - * - - * - - * - - *
                     |
beta:                * - - - - - - - - - *
```

Six weeks after the first beta was created, it's time for a stable release! The `stable` branch is produced from the `beta` branch:

```
nightly: * - - * - - * - - * - - * - - * - * - *
                     |
beta:                * - - - - - - - - - *
                                         |
stable:                                  *
```

Hooray! Rust 1.5 is done! However, we've forgotten one thing: because the six weeks have gone by, we also need a new beta of the *next* version of Rust, 1.6. So after `stable` branches off of `beta`, the next version of `beta` branches off of `nightly` again:

```
nightly: * - - * - - * - - * - - * - - * - * - *
                     |                         |
beta:                * - - - - - - - - - *       *
                                         |
stable:                                  *
```

This is called the "train model" because every six weeks, a release "leaves the station", but still has to take a journey through the beta channel before it arrives as a stable release.

Rust releases every six weeks, like clockwork. If you know the date of one Rust release, you can know the date of the next one: it's six weeks later. A nice aspect of having releases

scheduled every six weeks is that the next train is coming soon. If a feature happens to miss a particular release, there's no need to worry: another one is happening in a short time! This helps reduce pressure to sneak possibly unpolished features in close to the release deadline.

Thanks to this process, you can always check out the next build of Rust and verify for yourself that it's easy to upgrade to: if a beta release doesn't work as expected, you can report it to the team and get it fixed before the next stable release happens! Breakage in a beta release is relatively rare, but `rustc` is still a piece of software, and bugs do exist.

Maintenance time

The Rust project supports the most recent stable version. When a new stable version is released, the old version reaches its end of life (EOL). This means each version is supported for six weeks.

Unstable Features

There's one more catch with this release model: unstable features. Rust uses a technique called "feature flags" to determine what features are enabled in a given release. If a new feature is under active development, it lands on the main branch, and therefore, in nightly, but behind a *feature flag*. If you, as a user, wish to try out the work-in-progress feature, you can, but you must be using a nightly release of Rust and annotate your source code with the appropriate flag to opt in.

If you're using a beta or stable release of Rust, you can't use any feature flags. This is the key that allows us to get practical use with new features before we declare them stable forever. Those who wish to opt into the bleeding edge can do so, and those who want a rock-solid experience can stick with stable and know that their code won't break. Stability without stagnation.

This book only contains information about stable features, as in-progress features are still changing, and surely they'll be different between when this book was written and when they get enabled in stable builds. You can find documentation for nightly-only features online.

Rustup and the Role of Rust Nightly

Rustup makes it easy to change between different release channels of Rust, on a global or per-project basis. By default, you'll have stable Rust installed. To install nightly, for example:

```
$ rustup toolchain install nightly
```

You can see all of the *toolchains* (releases of Rust and associated components) you have installed with `rustup` as well. Here's an example on one of your authors' Windows computer:

```
> rustup toolchain list
stable-x86_64-pc-windows-msvc (default)
beta-x86_64-pc-windows-msvc
nightly-x86_64-pc-windows-msvc
```

As you can see, the stable toolchain is the default. Most Rust users use stable most of the time. You might want to use stable most of the time, but use nightly on a specific project, because you care about a cutting-edge feature. To do so, you can use `rustup override` in that project's directory to set the nightly toolchain as the one `rustup` should use when you're in that directory:

```
$ cd ~/projects/needs-nightly
$ rustup override set nightly
```

Now, every time you call `rustc` or `cargo` inside of *~/projects/needs-nightly*, `rustup` will make sure that you are using nightly Rust, rather than your default of stable Rust. This comes in handy when you have a lot of Rust projects!

The RFC Process and Teams

So how do you learn about these new features? Rust's development model follows a *Request For Comments (RFC) process*. If you'd like an improvement in Rust, you can write up a proposal, called an RFC.

Anyone can write RFCs to improve Rust, and the proposals are reviewed and discussed by the Rust team, which is comprised of many topic subteams. There's a full list of the teams on Rust's website, which includes teams for each area of the project: language design, compiler implementation, infrastructure, documentation, and more. The appropriate team reads the proposal and the comments, writes some comments of their own, and eventually, there's consensus to accept or reject the feature.

If the feature is accepted, an issue is opened on the Rust repository, and someone can implement it. The person who implements it very well may not be the person who proposed the feature in the first place! When the implementation is ready, it lands on the main branch behind a feature gate, as we discussed in the "Unstable Features" section.

After some time, once Rust developers who use nightly releases have been able to try out the new feature, team members will discuss the feature, how it's worked out on nightly, and decide if it should make it into stable Rust or not. If the decision is to move forward, the feature gate is removed, and the feature is now considered stable! It rides the trains into a new stable release of Rust.